

Credit Card Fraud Detection

Architecture, Experiment Design & System Design

Project: github.com/Divy18s/credit-card-fraud-detection

Classical ML · Gradient Boosting · Deep Learning · GAN oversampling · GNNs · SHAP

September 2026

Contents

1	What was built	3
2	Technology stack	4
3	System architecture	5
3.1	End-to-end pipeline	5
3.2	The chronological split and the drift it preserves	5
3.3	Leakage-safe resampling: what happens at fit vs predict time	6
3.4	Code layout	6
4	Step-by-step: technology, function, handling	6
4.1	Step 1 – Features	6
4.2	Step 2 – Chronological split	7
4.3	Step 3 – Imbalance strategies	7
4.4	Step 4 – Models	7
4.5	Step 5 – Threshold and metrics	7
4.6	Step 6 – Bootstrap, ablation, SHAP, app	7
5	Deep-dive: handling 0.17% fraud	7
5.1	The techniques	7
5.2	Results grid (test PR-AUC, best per strategy in bold)	8
6	Deep-dive: why a chronological split	8
6.1	The validity argument	8
6.2	Measured effect (same code, only the split assignment changes)	8
6.3	Pre-registered replication of a published split	9
7	Deep-dive: how much can 52 test frauds tell us?	9
7.1	PR-AUC and the bootstrap	9
8	Results	10

8.1	Best configuration of each model type (test window, 52 frauds)	10
8.2	What the grid says	10
8.3	Champion operating point	11
9	Explainability (SHAP)	11
10	Audit: what was checked, and what was found	12
10.1	Recomputed from the raw data and saved artifacts	12
10.2	Issues found (none changes a headline result)	12
11	System design decisions (interview-ready)	13
11.1	Likely interview questions	14
12	Limitations and future work	14
13	Resume entry	14
13.1	Plain text for copy-paste	15
14	Thirty-second pitch	15
A	Reproduce	15

1 What was built

Card fraud is rare, expensive to miss, and changes over time. The Kaggle ULB dataset used here has 284,807 card transactions over 48 hours with only **492 frauds (0.173%)**. Its features are 28 anonymised PCA components (V1–V28) plus **Time** and **Amount**. There are no cardholder, merchant or device IDs.

The project is a comparative study rather than a single model: **72 training runs** over classical ML, gradient boosting, deep learning, GAN oversampling, ensembles and graph neural networks, all under *one* evaluation protocol designed to mirror deployment. It ships with:

- **A chronological train / validation / test split** (70/15/15 by time): models learn from the past and are scored on the future, exactly as a deployed model would be.
- **Honest model selection:** resampling, class weights, hyperparameters, early stopping and decision thresholds are all chosen on validation; the test window is scored once.
- **Uncertainty, not just rankings:** a stratified bootstrap shows which differences are real.
- **Explainability and a demo:** SHAP explanations of the champion, served in a Streamlit app.
- **Ablations and a pre-registered replication** of a published benchmark.

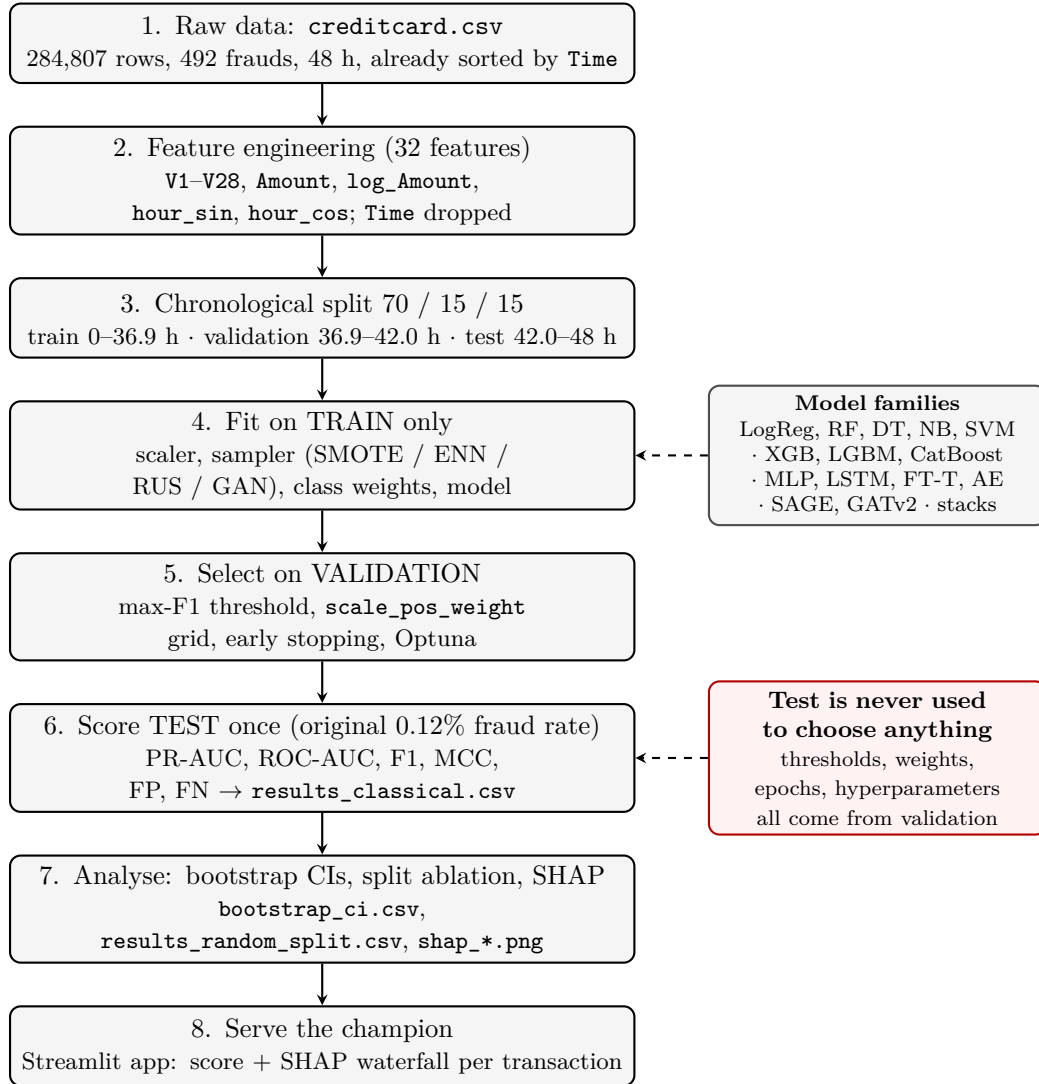
Concern	Common approach	This project
Split	random stratified shuffle	chronological 70/15/15 (train on past, test on future)
Resampling	SMOTE applied to the whole dataset before splitting	inside an imblearn pipeline: fitted on training rows only
Threshold	0.5, or tuned on test	max-F1 threshold chosen on validation, frozen before test
Metric	accuracy / ROC-AUC	PR-AUC (average precision) first; F1, MCC, FP/FN counts
Class weight	$w = \text{imbalance ratio}$ (518)	selected on validation from a grid (ratio collapses LightGBM)
Comparison	single leaderboard	1,000× stratified bootstrap CIs; paired tests
Model scope	one family	18 model types, 8 imbalance strategies, GAN, stacking, GNNs, Optuna
Trust	results as reported	ablations, a pre-registration, documented errata, SHAP

2 Technology stack

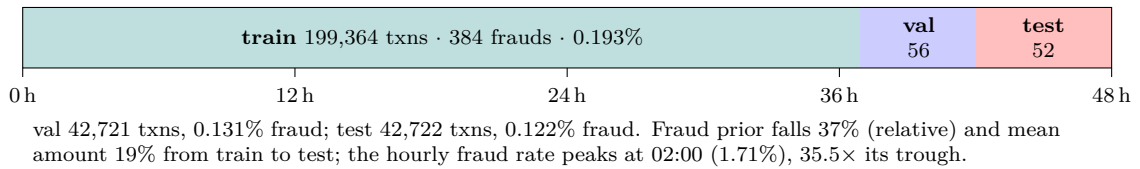
Layer	Technology	Role
Language	Python 3.12	Everything
Data	pandas, NumPy	Loading, feature engineering, chronological splitting
Classical ML	scikit-learn 1.9	Logistic regression, Random Forest, Decision Tree, Naive Bayes, calibrated LinearSVC
Boosting	XGBoost 3.4 (CUDA), LightGBM 4.7, CatBoost 1.2 (GPU)	Gradient-boosted trees with validation-selected scale_pos_weight
Imbalance	imbalanced-learn 0.14	SMOTE, SMOTE-ENN, random undersampling inside imblearn.Pipeline
Deep learning	PyTorch 2.13 (CUDA 13, RTX 5050)	MLP, deep MLP, AutoEncoder, LSTM/BiLSTM, FT-Transformer, tabular GAN
Graphs	PyTorch Geometric 2.8	GraphSAGE, GATv2 on k -NN transaction graphs
Tuning	Optuna (TPE, seed 42)	RF / XGBoost / LightGBM / CatBoost / GraphSAGE search on validation PR-AUC
Explainability	SHAP 0.52 (TreeExplainer)	Global and per-transaction attributions for the champion
App	Streamlit	Single-transaction scoring + SHAP waterfall, batch CSV scoring, leaderboard
Reporting	Jupyter, matplotlib, L ^A T _E X	EDA and final notebooks, report PDFs, this document

3 System architecture

3.1 End-to-end pipeline

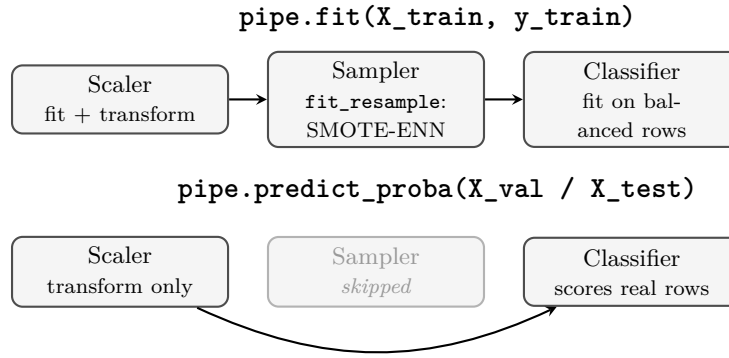


3.2 The chronological split and the drift it preserves



Sorting by time and cutting at 70% and 85% means the model is always fitted on the past and scored on the future. A random shuffle would erase this drift by making all three partitions look alike.

3.3 Leakage-safe resampling: what happens at fit vs predict time



`imblearn.Pipeline` applies samplers only during `fit`. Validation and test rows are never resampled, so metrics are measured on the real 0.12% fraud rate. Resampling *before* splitting, a common error in published work, would let synthetic copies of test frauds into training.

3.4 Code layout

File	Responsibility
<code>src/config.py</code>	Paths, seed 42, split fractions
<code>src/data/dataset.py</code>	Load CSV, add features, chronological / random / Sang splits
<code>src/models/metrics.py</code>	Validation max-F1 threshold; PR-AUC, ROC-AUC, F1, MCC, confusion counts
<code>src/models/train_classical.py</code>	8 classical models $\times$ 7 imbalance strategies; <code>scale_pos_weight</code> grid on validation
<code>src/models/gan.py</code>	Tabular GAN for synthetic frauds; generator cached by a hash of the training frauds
<code>src/models/deep.py, train_deep.py</code>	MLP, DNN, AutoEncoder, LSTM / BiLSTM (8-step windows), FT-Transformer; early stopping on validation
<code>src/models/gnn.py, train_gnn.py</code>	k -NN graphs per split; GraphSAGE (full batch), GATv2 (sampled subgraphs)
<code>src/models/train_ensembles.py</code>	5-seed LSTM ensemble; RF + MLP + LSTM average
<code>src/models/tune_optuna.py</code>	TPE search on validation PR-AUC
<code>src/models/ablation_random_split.py</code>	Same configs under chronological vs random split
<code>src/models/bootstrap_ci.py</code>	Saves test score vectors; stratified bootstrap CIs
<code>src/models/explain_shap.py</code>	Trains + saves the champion bundle; SHAP figures and summary
<code>app/streamlit_app.py</code>	Demo app
<code>reports/</code>	Ledgers (CSV), score vectors (<code>.npz</code>), figures, report PDFs, pre-registration

4 Step-by-step: technology, function, handling

4.1 Step 1 – Features

Tech: `pandas`, `NumPy` (`add_features`).

Does: adds `log_Amount` (tames the heavy-tailed amount) and a cyclical hour encoding ($\sin \frac{2\pi h}{24}, \cos \frac{2\pi h}{24}$), so 23:00 and 00:00 are neighbours. `Time` itself is dropped: a raw timestamp would let models memorise *when* a window was, which can't generalise.

Handles: 32 model features in a fixed order, stored with the champion bundle so the app can't feed columns in the wrong order.

4.2 Step 2 – Chronological split

Tech: `time_split`: sort by `Time`, cut at 70% and 85%.

Does: train on hours 0–36.9, choose on 36.9–42.0, test on 42.0–48.

Handles: exact duplicate rows share a `Time` value, so they stay together (0 cross the boundary). The same module also builds the random split for the ablation, and Sang’s 66/14/20 split, which reproduces that paper’s partition to the row (187,972 / 39,873 / 56,962; 368 / 49 / 75 frauds).

4.3 Step 3 – Imbalance strategies

Tech: imbalanced-learn samplers inside `imblearn.Pipeline`; a PyTorch GAN.

Does: eight ways to handle 518:1 imbalance: do nothing; random undersampling to 1:1; SMOTE to 1:1; SMOTE followed by Edited Nearest Neighbours cleaning; class weights (validation-selected, or fixed at the ratio); and GAN-generated frauds to 1:1.

Handles: samplers only ever see training rows (Section 3.3). For boosted trees, `scale_pos_weight` is picked on validation from $\{1, 4, 16, \sqrt{r}, 64, r\}$, which prevents the collapse that the textbook $w = r$ causes (LightGBM: PR-AUC 0.009 with 4,894 false positives).

4.4 Step 4 – Models

Tech: scikit-learn, XGBoost/CatBoost on CUDA, LightGBM on CPU, PyTorch, PyTorch Geometric.

Does: 18 model types. Sequence models see the last 8 transactions as a window; graph models see a 10-nearest-neighbour graph built separately inside each split; the FT-Transformer tokenises each feature; the AutoEncoder learns legitimate traffic only and flags high reconstruction error.

Handles: neural nets and GNNs early-stop on validation PR-AUC; seeds fixed at 42.

4.5 Step 5 – Threshold and metrics

Tech: `best_threshold_f1` (validation), `evaluate_scores` (test).

Does: the threshold maximising F1 on validation is frozen, then test is scored once.

Handles: thresholds in the ledger range from 0.07 to 1.0; a fixed 0.5 would be badly wrong for most models. PR-AUC is the headline because its baseline equals the fraud rate (0.0012 on test), whereas ROC-AUC looks excellent (> 0.95) even for mediocre fraud detectors.

4.6 Step 6 – Bootstrap, ablation, SHAP, app

Tech: saved score vectors (`.npz`); stratified bootstrap; `TreeExplainer`; Streamlit.

Does: turns single numbers into intervals; measures what the split protocol is worth; explains the champion; serves it.

Handles: the app samples only from the held-out test window (rows 242,085–284,806, verified identical to the training code’s test split) and loads a bundle of {pipeline, threshold, feature order}.

5 Deep-dive: handling 0.17% fraud

5.1 The techniques

- **SMOTE** creates synthetic frauds on the line between a fraud x_i and one of its $k = 5$ nearest fraud neighbours x_{nn} : $x_{\text{new}} = x_i + \lambda(x_{nn} - x_i)$, $\lambda \sim U(0, 1)$.
- **ENN** then deletes any row whose class disagrees with the majority of its 3 nearest neighbours, cleaning the noisy boundary SMOTE tends to create.

- **Class weighting** multiplies the loss on frauds by w (`scale_pos_weight`, `pos_weight`); no rows are added.
- **GAN**: a generator maps noise $z \sim \mathcal{N}(0, I_{24})$ to synthetic fraud rows while a discriminator learns to tell them from real frauds (label smoothing 0.9, 8,000 steps). The cache key is a SHA-1 of the training frauds, so a generator trained on one split is never reused on another.

5.2 Results grid (test PR-AUC, best per strategy in bold)

Strategy	LogReg	Random Forest	XGBoost	LightGBM	CatBoost
raw	0.649	0.749	0.714	0.537	0.752
random undersampling	0.627	0.768	0.723	0.756	0.670
SMOTE	0.729	0.790	0.770	0.770	0.743
SMOTE-ENN	0.738	0.795	0.777	0.768	0.755
class weight (val-selected)	0.720	0.784	0.769	0.537	0.765
GAN	0.568	0.762	0.769	0.775	0.783

Hybrid SMOTE-ENN gives the champion (RF, 0.795). GAN synthesis is best for CatBoost and rescues LightGBM, but hurts the linear model; for sequences, weighting beats both SMOTE-ENN (0.758) and GAN (0.750) because interpolating time windows distorts their dynamics. Validation-selected weights are the fix for the collapse above: Optuna independently chose moderate weights too (4–172, never 518).

6 Deep-dive: why a chronological split

6.1 The validity argument

A random split trains on transactions that happen *after* the ones it is scored on. No deployed system can do that, so the number answers a question with no operational counterpart. That argument holds even if both protocols scored the same. Two measurable channels follow from shuffling:

1. **Temporal ordering destroyed**: late fraud patterns inform early predictions, and the drift of Section 3.2 disappears.
2. **Duplicates separated**: ULB has 1,081 extra copies of rows in 773 groups. Shuffling puts 517 of them across partition boundaries (249 train→test, 207 train→val, 61 val→test); the chronological split puts 0. Duplicates are also informative: 14.3% of €0.00 duplicates are fraud (card testing), against a 0.173% base rate.

A third channel, resampling before splitting, is excluded by construction (Section 3.3), so every effect measured here is a floor for what the protocol is worth in published work.

6.2 Measured effect (same code, only the split assignment changes)

Configuration	chronological	random	random – chronological
XGBoost + class weight	0.7688	0.8372	+0.0684
CatBoost + GAN	0.7804	0.8399	+0.0595
MLP raw	0.7794	0.8267	+0.0473
XGBoost + SMOTE-ENN	0.7766	0.8104	+0.0338
RF + SMOTE-ENN	0.7947	0.8120	+0.0173
LSTM + class weight	0.7803	0.7965	+0.0162
GraphSAGE + class weight	0.7389	0.6948	−0.0441
Pooled over 10 paired runs (7 at 70/15/15, 3 at 66/14/20)			+0.0182 , 7/10 positive

The pooled 95% CI is $[-0.0084, +0.0448]$ ($t = 1.55$; one-sided sign test $p = 176/1024 = 0.172$). The project reports this honestly: the data establish the *direction*, not a reliable magnitude. GraphSAGE is the exception: shuffling scrambles the temporal coherence of its neighbourhoods.

6.3 Pre-registered replication of a published split

Sang (2026) reports XGBoost PR-AUC 0.7902 on a 66/14/20 chronological split of the same CSV. The question, the three configurations and the reading were written down and committed before running:

Configuration	70/15/15	66/14/20	Δ split	vs 0.7902
XGBoost + class weight	0.7688	0.8055	+0.0367	+0.0153 (0.37 SE)
XGBoost + SMOTE-ENN	0.7766	0.8102	+0.0336	+0.0200 (0.48 SE)
RF + SMOTE-ENN	0.7947	0.8224	+0.0276	+0.0322 (0.77 SE)

Read as a replication, not an improvement: the closest analogue lands 0.37 SE from the published value ($SE \approx 0.050\sqrt{52/75} \approx 0.042$) and recovers the same 57 of 75 frauds. The bigger finding is that the cut point alone moved PR-AUC by +0.033 on average, more than the pooled protocol effect.

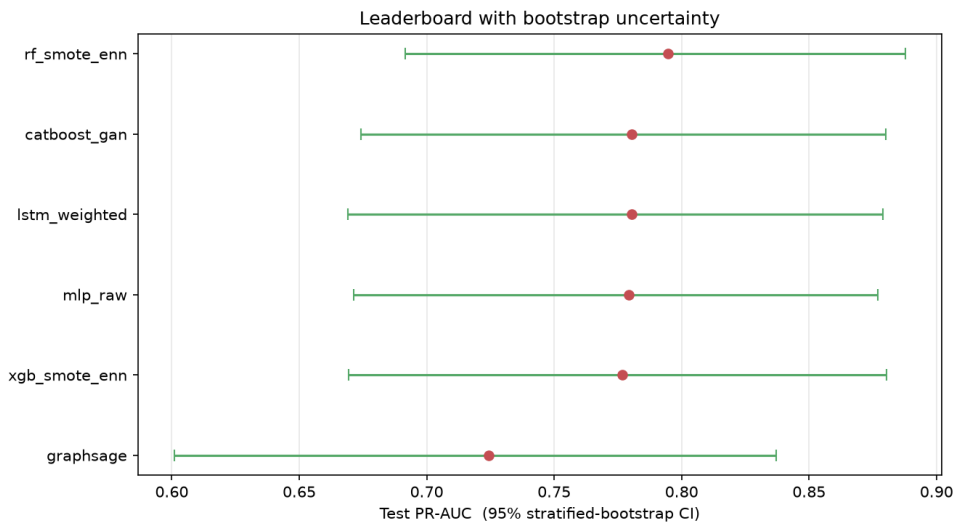
7 Deep-dive: how much can 52 test frauds tell us?

7.1 PR-AUC and the bootstrap

Average precision summarises the precision–recall curve over ranked scores:

$$AP = \sum_n (R_n - R_{n-1}) P_n, \quad \text{baseline (random scores)} = \frac{\# \text{frauds}}{\# \text{rows}} = \frac{52}{42,722} \approx 0.0012 \quad (1)$$

To get an interval, the test set is resampled 1,000 times *stratified*: the 52 frauds and the 42,670 legitimate rows are each resampled with replacement, so every replicate keeps the real class balance. The 2.5th and 97.5th percentiles of AP form the 95% CI.



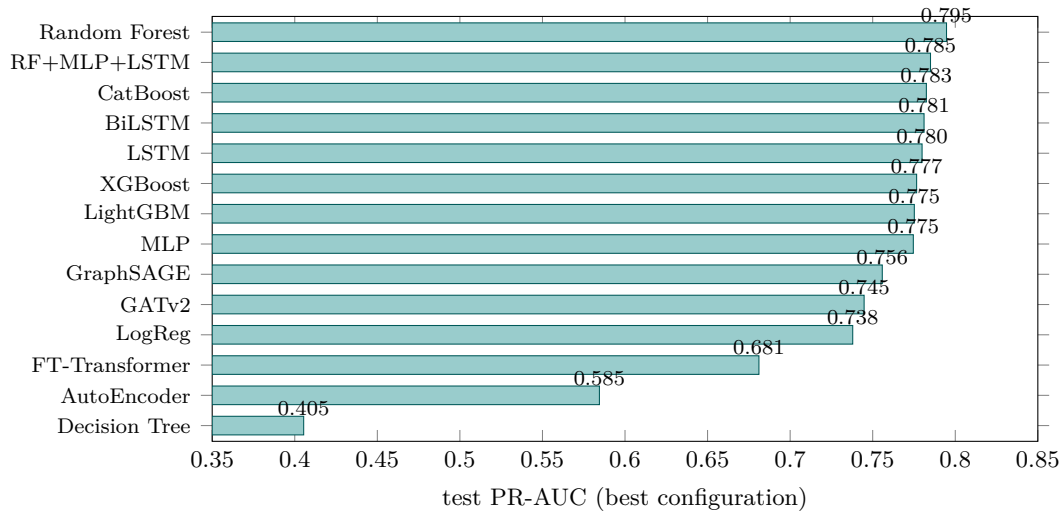
Representative config	PR-AUC	95% CI	width
RF + SMOTE-ENN	0.7947	[0.692, 0.888]	0.196
CatBoost + GAN	0.7804	[0.674, 0.880]	0.206
LSTM + class weight	0.7803	[0.669, 0.879]	0.210
MLP raw	0.7794	[0.671, 0.877]	0.206
XGBoost + SMOTE-ENN	0.7766	[0.670, 0.880]	0.211
GraphSAGE + class weight	0.7242	[0.601, 0.837]	0.236

All 15 pairwise intervals overlap. A paired bootstrap of RF vs XGBoost (same resamples for both) gives a difference of +0.018 with a CI that includes zero. And one transaction is worth about ± 0.020 : promoting the worst-ranked test fraud to the top moves PR-AUC by +0.0198, demoting the best-ranked to the bottom by -0.0199 . The honest output is a shared band of roughly 0.72–0.89, *not* a ranking.

8 Results

8.1 Best configuration of each model type (test window, 52 frauds)

Model	Strategy	PR-AUC	ROC-AUC	F1	MCC	FP / FN
Random Forest	SMOTE-ENN	0.7947	0.971	0.848	0.855	1 / 13
RF + MLP + LSTM	stacked average	0.7850	0.984	0.835	0.844	1 / 14
CatBoost	GAN	0.7825	0.972	0.848	0.855	1 / 13
BiLSTM	class weight	0.7811	0.980	0.839	0.844	2 / 13
LSTM	class weight	0.7799	0.971	0.813	0.821	2 / 15
XGBoost	SMOTE-ENN	0.7766	0.973	0.812	0.815	5 / 13
LightGBM	GAN	0.7752	0.981	0.822	0.832	1 / 15
MLP	raw	0.7746	0.971	0.809	0.821	1 / 16
GraphSAGE	class weight	0.7558	0.923	0.796	0.801	4 / 15
GATv2	class weight	0.7448	0.966	0.783	0.789	4 / 16
Logistic regression	SMOTE-ENN	0.7379	0.975	0.767	0.785	1 / 19
FT-Transformer	class weight	0.6811	0.980	0.760	0.760	10 / 14
AutoEncoder	anomaly score	0.5845	0.941	0.660	0.660	19 / 17
Decision Tree	class weight	0.4054	0.855	0.632	0.636	28 / 15



8.2 What the grid says

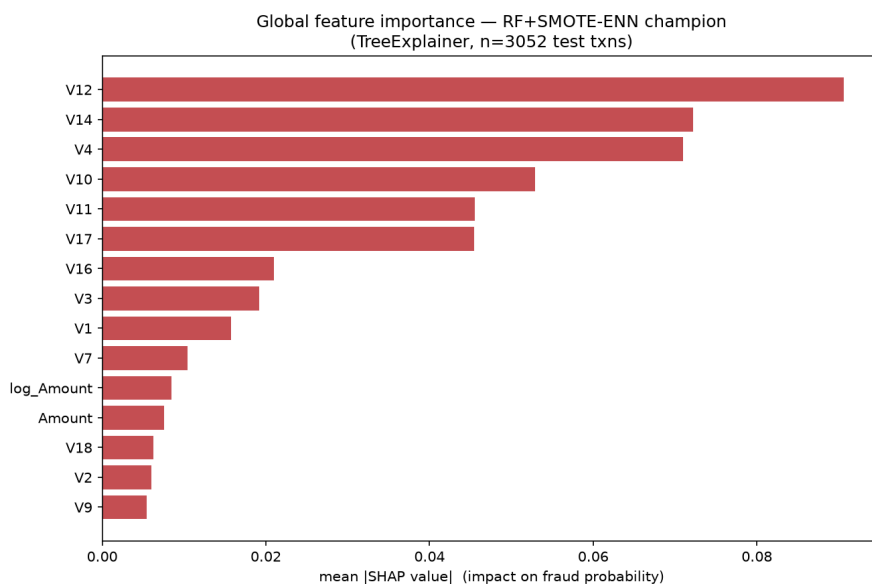
- **Ensembles of trees win; a single tree fails.** RF 0.795 vs its own Decision Tree 0.405.

- **Diversity beats seeds.** RF + MLP + LSTM (0.785) beats every deep model; a 5-seed LSTM ensemble (0.775) doesn't beat its own best member.
- **Depth and attention don't help on 32 PCA features.** A 512-256-128-64 DNN ties the small MLP (0.7752 vs 0.7746); the FT-Transformer trails both.
- **Graphs without entities $\approx$ an MLP.** A feature-similarity graph carries signal (SAGE 0.756) but no real relational structure; published graph results (≈ 0.89) need cardholder and merchant IDs, which this dataset doesn't have.
- **Tuning repairs, it doesn't lift.** Optuna took LightGBM from 0.537 to 0.753 and XGBoost raw from 0.714 to 0.769, but every family's best stays within ± 0.01 .
- **Unsupervised isn't enough.** The AutoEncoder reaches ROC-AUC 0.94 but PR-AUC only 0.58.

8.3 Champion operating point

At its validation-chosen threshold (0.718) the champion flags 40 of 42,722 test transactions: **39 frauds and 1 legitimate**, missing 13. That is 97.5% precision at 75% recall. In an operations setting, nearly every alert an analyst opens is real.

9 Explainability (SHAP)



TreeExplainer on 3,052 test rows (all 52 frauds + 3,000 random legitimate). Top drivers: V12, V14, V4, V10, V11, V17. Low V12/V14/V10/V17 and high V4/V11 push toward fraud, matching the signs of the EDA correlations. Shapley values decompose each prediction additively, $f(x) = \phi_0 + \sum_j \phi_j$, which is what the app's per-transaction waterfall shows.

10 Audit: what was checked, and what was found

10.1 Recomputed from the raw data and saved artifacts

Claim	Recomputed	Status
Dataset: 284,807 rows, 492 frauds, 0.173%	284,807 / 492 / 0.173%	✓
Split sizes, frauds, hours, fraud rates, mean amounts	all 15 values identical	✓
Prior −37%, amount −19%, peak 02:00 at 1.71%, 35.5×	−36.8%, −19.2%, 02:00 / 1.71%, 35.5×	✓
1,081 duplicates in 773 groups; 517 leak randomly, 0 chronologically	1,081 / 773; 249 + 207 + 61 = 517; 0	✓
Card-testing rates 14.3% (€0) and 4.6% (€0.01–2)	14.3% and 4.6%	✓
Sang splits reproduce exactly; shuffled test has 98 frauds	187,972 / 39,873 / 56,962; 368 / 49 / 75; 98	✓
Champion: PR-AUC 0.795, F1 0.848, MCC 0.855, 1 FP, 13 FN	saved model re-scored: 0.7947, 0.8478, 0.8550, 1, 13	✓
Champion reproduces from code	retrained from scratch (169 s): identical metrics and threshold; scores within 2.2×10^{-16} of the saved vector	✓
Bootstrap CIs (6 configs) and “all 15 pairs overlap”	all 6 CIs identical to 4 d.p.; 15 / 15 overlap	✓
One transaction $\approx \pm 0.020$ PR-AUC	+0.0198 / −0.0199; single-fraud removal range 0.0203	✓
XGBoost + class weight: 0.7688 chronological / 0.8372 random	re-run: 0.7688 (spw 16, 2 FP / 16 FN) / 0.8372	✓
Pooled +0.0182, 7/10 positive, sign test $p = 0.172$	recomputed from the three ledgers	✓
Tuned results (LGBM 0.753, XGB 0.769, RF 0.782, CatBoost 0.766)	present in the ledger	✓
Pre-registration committed before the replication run	04:04:41 pre-registration, 04:16:29 result (backup branch)	✓
Demo app loads, scores, explains	served headless; single-transaction and leaderboard tabs render	✓

10.2 Issues found (none changes a headline result)

1. **GATv2 early stopping uses the training graph.** `train_gat_minibatch` computes its “validation” AP on the same graph it trains on; the docstring says validation. Measured in a scratch re-run: shipped code 0.7684 test PR-AUC (1-hop inference) / 0.7698 (exact 2-hop inference); corrected (early stopping on the validation graph) 0.7329; ledger 0.7448. GATv2 lands at 0.73–0.77 every way, so “attention adds cost, not accuracy” stands, but the row is one draw from a spread of 0.037 (0.733–0.770).
2. **GAT inference comment is inaccurate.** `predict_minibatch` says “no edges are lost”, but a 2-layer model gets only 1-hop context. The effect is tiny (0.7684 vs 0.7698).
3. **Graph models see future features within a window.** Each split’s k -NN graph links a transaction to later transactions in the same window (features only, never labels). A deployed system couldn’t see those. It affects GraphSAGE and GATv2 only.
4. **Duplicate leakage is understated.** The 517 counts byte-identical rows including `Time`, which isn’t a model input. Rows identical in the 32 features the models actually see: **1,784** under the random split vs 8 chronologically (0 train→test, none of them frauds). This strengthens the project’s argument.
5. **Paired RF vs XGBoost test isn’t in the code.** README: +0.0187, CI [−0.0119, +0.0515]. Recomputed from the saved scores: point +0.0181 (bootstrap mean

+0.0188), CI $[-0.0116, +0.0510]$. Same conclusion; the script should be committed and the point estimate quoted.

6. **Two results aren't in any file.** The XGBoost + class weight random-split run (0.8372) and the Optuna parameters of the four boosters (`optuna_best_params.json` holds only GraphSAGE). The first was reproduced exactly by re-running; the second can't be traced.
7. **The public history can't show the pre-registration.** `main` on GitHub was squashed into one commit; the ordering evidence exists only on the local branch `backup-main-history`.
8. **Maintenance.** Streamlit's `use_container_width` is deprecated (warned for removal after 2025-12-31); `ablation_random_split.run_gnn` hard-codes "cuda", so it fails on a CPU-only machine.

11 System design decisions (interview-ready)

1. **Evaluate the way you deploy.** Train on the past, test on the future; select on a window in between. Any other protocol answers a question production never asks.
2. **Put every data-dependent step inside the pipeline.** Scalers and samplers are fitted with the model on training rows only, so resampling can't leak.
3. **Test is read once.** Thresholds, class weights, epochs and hyperparameters come from validation. The one "fixed-weight" variant is kept as a documented failure, not tuned away.
4. **Rank with PR-AUC, report with counts.** At 0.12% prevalence ROC-AUC flatters everything; PR-AUC plus FP/FN counts is what an operations team can use.
5. **Intervals before rankings.** With 52 test frauds, one transaction is worth ± 0.02 ; claims smaller than that are noise.
6. **Fingerprint cached artifacts.** The GAN cache is keyed by a hash of the training frauds after an early run reused one generator across protocols (documented as an erratum).
7. **Ship the model with its contract.** The joblib bundle holds pipeline, threshold and feature order together.

11.1 Likely interview questions

Question	Answer
Why not accuracy or ROC-AUC?	Predicting “legit” for everything is 99.88% accurate. ROC-AUC counts true negatives, which are abundant, so even weak models score above 0.95. PR-AUC’s baseline is the prevalence (0.0012), so it measures what matters: how clean the top of the ranking is.
Why SMOTE inside the pipeline?	If you oversample first and split later, synthetic neighbours of test frauds end up in training; the model is graded on near-copies of what it learned.
Why did $w = 518$ break LightGBM?	Huge weights make every leaf split chase the positives; the model predicted fraud on 4,894 legitimate test rows. Selecting w on validation (it chose 1) avoids it.
Is RF really the best?	It has the highest point estimate, but its CI overlaps every other tier’s; the paired test against XGBoost includes zero. The defensible claim is a band, not a winner.
Why is the random split higher?	Shuffling mixes future patterns into training and separates duplicates across the boundary; it also erases drift. The pooled gain is +0.018 with a CI including zero, so the magnitude is uncertain, but the protocol is invalid regardless.
How would this change in production?	Add real entities (card, merchant, device) for velocity features and genuine graphs; retrain on a rolling window; calibrate scores; monitor precision at the alert threshold as fraud drifts; and account for label delay (chargebacks arrive weeks later).

12 Limitations and future work

- **48 hours, 52 test frauds.** Every comparison is statistically loose; a longer dataset is the real fix.
- **Anonymised PCA features, no entity IDs.** Rules out velocity features and true relational graphs.
- **Single chronological cut.** Rolling-origin (walk-forward) evaluation would show how stable the ranking is over time.
- **Run-to-run variance** for neural and graph models (0.01–0.04) is larger than several leaderboard gaps; multi-seed means would be more robust.
- **No cost model.** Threshold selection maximises F1; a production system would weigh the cost of a missed fraud against the cost of an analyst review.

13 Resume entry

Credit Card Fraud Detection | *Python, scikit-learn, XGBoost, PyTorch, PyTorch Geometric* ([GITHUB](#)) Aug 2026

- Benchmarked **72 fraud-detection pipelines** across classical ML, gradient boosting, LSTM/Transformer, GAN oversampling and GNNs on **284,807 transactions (0.17% fraud)** under a chronological split with in-pipeline resampling and validation-only tuning.
- Delivered a **Random Forest + SMOTE-ENN** champion with **PR-AUC 0.795 and F1 0.848**, reaching **97.5% precision at 75% recall with 1 false positive in 42,722** future transactions, served in a **Streamlit** app with per-prediction **SHAP** explanations.
- Quantified uncertainty with a **1,000× stratified bootstrap**: 95% CIs span **0.20–0.24** and **all 15 model pairs overlap**, with one test fraud worth ± 0.020 PR-AUC, so results are reported as a performance band rather than a false ranking.
- Showed random shuffling leaks **517 duplicate rows** across partitions (0 chronologically) and **pre-registered a replication** of a published benchmark, landing **0.37 SE** from its PR-AUC (0.8055 vs 0.7902).

13.1 Plain text for copy-paste

Select the box and copy. No hyphenation or ligatures, so words paste intact; the same text is in `docs/resume_entry.txt`.

Credit Card Fraud Detection | Python, scikit-learn, XGBoost, PyTorch, PyTorch Geometric (GITHUB)
— Aug 2026

- Benchmarked 72 fraud-detection pipelines across classical ML, gradient boosting, LSTM/Transformer, GAN oversampling and GNNs on 284,807 transactions (0.17% fraud) under a chronological split with in-pipeline resampling and validation-only tuning.
- Delivered a Random Forest + SMOTE-ENN champion with PR-AUC 0.795 and F1 0.848, reaching 97.5% precision at 75% recall with 1 false positive in 42,722 future transactions, served in a Streamlit app with per-prediction SHAP explanations.
- Quantified uncertainty with a 1,000× stratified bootstrap: 95% CIs span 0.20–0.24 and all 15 model pairs overlap, with one test fraud worth ± 0.020 PR-AUC, so results are reported as a performance band rather than a false ranking.
- Showed random shuffling leaks 517 duplicate rows across partitions (0 chronologically) and pre-registered a replication of a published benchmark, landing 0.37 SE from its PR-AUC (0.8055 vs 0.7902).

14 Thirty-second pitch

“I compared 72 fraud-detection pipelines, from logistic regression to LSTMs, GANs and graph neural networks, on 284,807 card transactions where only 0.17% are fraud. The key decision was the protocol: train on the past and test on the future, with resampling and thresholds chosen only on validation, because that’s how a deployed model is used. The best model, a Random Forest with SMOTE-ENN, catches three quarters of frauds at 97.5% precision, one false alarm in 42,722 transactions, and every prediction is explained with SHAP in a Streamlit app. But I also bootstrapped the test set and found all the top models’ intervals overlap, so the honest result is a performance band, not a winner.”

A Reproduce

```
python -m venv .venv && .venv\Scripts\activate && pip install -r requirements.txt
python -m src.data.make_dataset # download creditcard.csv (Kaggle token)
python -m src.models.train_classical # classical + boosting grid
python -m src.models.train_deep # MLP / LSTM / FT-Transformer / AutoEncoder
python -m src.models.train_gnn # GraphSAGE / GATv2
python -m src.models.train_ensembles # LSTM ensemble, RF+MLP+LSTM stack
python -m src.models.tune_optuna # Optuna tuning
python -m src.models.ablation_random_split # chronological vs random
python -m src.models.train_classical --split sang # pre-registered replication
python -m src.models.bootstrap_ci # CIs from saved score vectors
python -m src.models.explain_shap # champion bundle + SHAP figures
.venv\Scripts\streamlit run app/streamlit_app.py # demo
```